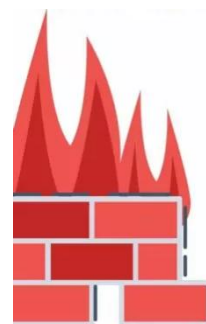


Perimeter Protection

Introduction

- ✓ Perimeter defense refers to the set of techniques and technologies that ensure that there are no intruders within the network boundary.
- ✓ Security uses technologies such as:
 - Firewalls
 - *Border routers*
- ✓ Evolution of perimeter defense
 - Originally, defense was simply performed using a router that allowed traffic to be confined to a single subnet (*Border router*).
 - The term firewall appears derived from the wall responsible for confining fires to a single building, preventing their spread. First appears in the film "War Games"



Perimeter Protection

- ✓ From this point on, the firewall evolves rapidly:
 - Packet inspection or stateless firewall (1988) – The firewall checks each of the packets transmitted between packets and, based on a series of rules, rejects or accepts the packets. The most common rules are the type of protocol and ports of origin and/or destination
 - Firewall by connection or *statefull* logging (1989) – The firewall keeps track of connections and can authorize or reject connections in addition to the existence of *stateless firewall rules*.
 - Firewall with *application layer* inspection (1991) – The firewall has the ability to inspect traffic and apply rules to the applications used.
 - Full Firewall, Unified Threat Management (2004) – Firewall with inspection at *application layer* in which anti-virus, anti-spam, etc. is added
 - *Next generation firewall* (2009) - *statefull* firewall in which features such as *Application Control, URL Filtering, Advanced Malware Protection, Network Profiling, Identity Policy* are added.

Perimeter Protection

- *Firewall as a service* (2018) – Cloud-resident NGFW firewall

Stateless firewall

- ✓ The firewall works on packet filtering by inspecting the *packet header*.
- ✓ The *header* has the following information: protocol, source IP address, destination IP address, port. The firewall allows, blocks, or ignores a particular packet based on rules that affect one of these fields.
- ✓ Example:
 - Accepts overseas traffic.
 - `iptables -t raw -I OUTPUT -j NOTRACK`
 - Ignore ping requests coming from subnet 192.168.1.0/24
 - `iptables -A INPUT -p icmp -d 192.168.1.0/24 --icmp-type 8 -j DROP`

Perimeter Protection

Stateful firewall

- ✓ The firewall works with rules identical to the *stateless firewall*.
- ✓ The firewall keeps track of connections. If the connection was allowed, it is still allowed.
- ✓ The *header* has the following information: protocol, source IP address, destination IP address, port. The firewall allows, blocks, or ignores a particular packet based on rules that affect one of these fields.
- ✓ Example:
 - Accept existing outbound connections.
 - `iptables -A OUTPUT -m conntrack --ctstate ESTABLISHED -j ACCEPT`
 - accept SSH connections
 - `iptables -A INPUT -p tcp --dport 22 -m conntrack --ctstate NEW,ESTABLISHED -j ACCEPT`

Firewall Testing

Simple Testing – SCAN (often using nmap)

TCP SYN scan – Take advantage of the fact that RFC793 "forces" a negative response (with an RST-reset packet) on an unforeseen TCP connection request. A properly configured firewall should not respond to SYN requests.

TCP ACK scan – Take advantage of the fact that RFC793 "forces" a negative response (with an RST-reset packet) to packets with the ACK flag at "1" without a prior connection. A properly configured firewall should not respond to ACK packets without having accepted SYN requests.

UDP scan – Since the UDP protocol does not imply a connection, this scan is of more dubious results. Usually the scan checks for response to well-known UDP services. Ex: DNS

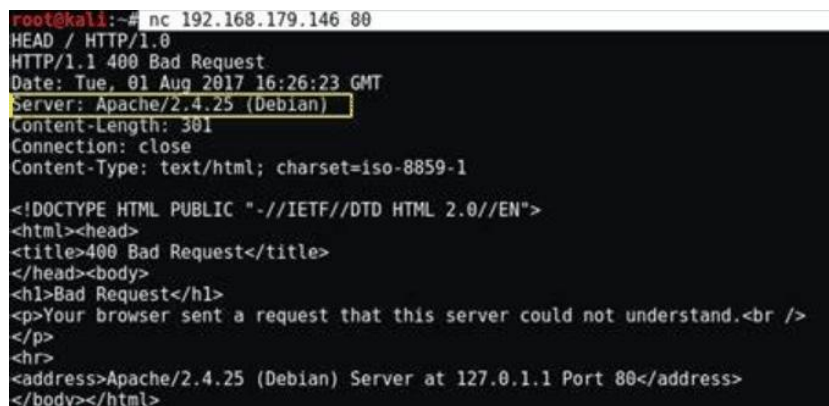
Perimeter Protection

Complex Tests – PEN (Waterfall Testing)

1st test – Check if there is a firewall, for example using traceroute to the domain.

2nd test – Verify which services are accessible (TCP SYN, UDP scan)

3rd test – Check the versions of the services (using netcat)



```
root@kali:~# nc 192.168.179.146 80
HEAD / HTTP/1.0
HTTP/1.1 400 Bad Request
Date: Tue, 01 Aug 2017 16:26:23 GMT
Server: Apache/2.4.25 (Debian)
Content-Length: 301
Connection: close
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>400 Bad Request</title>
</head><body>
<h1>Bad Request</h1>
<p>Your browser sent a request that this server could not understand.<br />
</p>
<hr>
<address>Apache/2.4.25 (Debian) Server at 127.0.1.1 Port 80</address>
</body></html>
```

4th test – Verify the network topology by testing with different TTL (time to live)

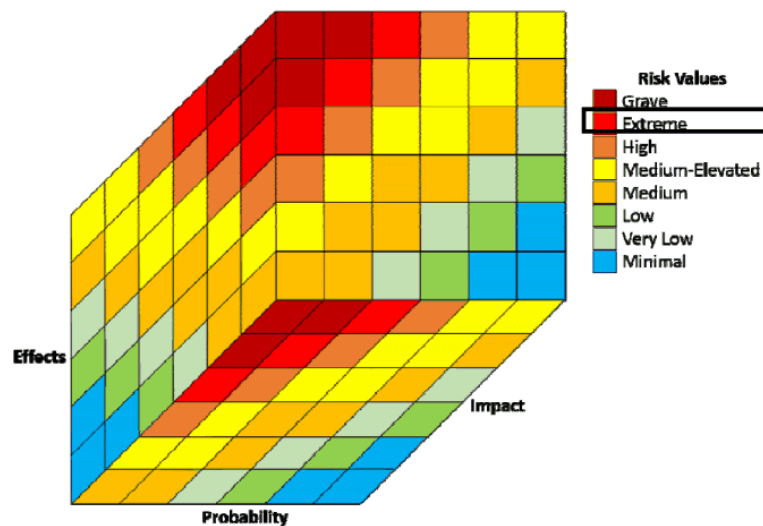
5th test – Port redirection. Check if a port is not at the default value (http port 80 is on port 3456). These tests are performed with the fpipe and datapipe tools

Perimeter Protection

6th test – Check for backdoors.

7th test – HTTP and HTTPS tunneling. Check if other services cannot be accessed over HTTP and HTTPS.

8th test – etc



<https://purplesec.us/firewall-penetration-testing/>

Perimeter Protection

AI-based pen testing

- ✓ It uses MML and autonomous agents to identify vulnerabilities, including:
 - Logic flaws in the business model. Traditional scanners rarely detect.
 - Simulation of real attacks autonomously.
 - Real-time analysis of patterns and responses to attacks.
 - Testing AI systems with attacks such as prompt injection and data poisoning.
 - Vulnerability prioritization

Perimeter Protection

Tool	Primary Focus	AI/Automation Level	Best For
ZeroThreat.ai	AI-driven web & API pentesting	High (AI-led attack simulation)	Startups, SaaS, DevSecOps teams
Burp Suite	Web app security testing	Medium (scanner + manual)	Security teams & pentesters
Nessus	Vulnerability scanning (network & systems)	High (automated scanning)	IT, security & compliance teams
Metasploit	Exploit development & validation	Medium (framework automation)	Security researchers & red teams
Rapid7	Automated DAST (InsightAppSec)	High (automated scanning)	Enterprises & compliance teams
OWASP ZAP	Open-source DAST	Medium (automated + manual)	Developers & SMBs
Traceable	API security & runtime protection	High (behavioral AI)	API-first companies
Terra Security	Automated web pentesting	High (AI automation)	Growing SaaS companies

<https://secureblitz.com/top-ai-powered-automated-pentesting-tools>

Host Info

Perimeter Protection

Example of pentest result with AI, firewall protected host

Prompt: scan 192.168.3.3

- **Hostname:** shellyplus2pm-a0dd6c28f37c
 - **Device:** Shelly Plus 2PM — a smart power relay/energy monitor
-

Ports Found

Port	State	Service	Notes
53/tcp	Open	DNS	NLnet Labs NSD
80/tcp	Filtered	HTTP	Blocked by firewall/ACL

Notes

- This is an **IoT device** (Shelly Plus 2PM smart switch), which is a common home automation component.
- Port **80 is filtered** — meaning a firewall is dropping packets rather than rejecting them, suggesting there's some network-level protection in front of it, which is good practice for IoT devices.
- Shelly devices run a built-in web UI on port 80 for configuration — keeping it firewalled off while managing it only through Home Assistant or the Shelly Cloud is the right approach.
- Make sure the device's **firmware is up to date** and that it's **not directly exposed to the internet**.

Perimeter Protection

Prompt: give me more information about this host

Let me run a fuller scan — wider port range, more aggressive service detection, and pull the HTTP content from port 80.

Excellent — the mDNS/Zeroconf service advertisement gave us detailed info. Here's the full picture:

 Full Device Profile — 192.168.3.3

Field	Value
Device	Shelly Plus 2PM
Hostname	ShellyPlus2PM-A0DD6C28F37C
Generation	Gen 2
App	Plus2PM
Firmware	1.7.4
MAC	A0:DD:6C:28:F3:7C
Web UI	Port 80/tcp (filtered from this network)
mDNS/Zeroconf	5353/UDP — Open and Advertising

 Open Services Summary

Port	Proto	State	Service
53/tcp	TCP	Open	DNS (NSD)
80/tcp	TCP	Filtered	HTTP Web UI

Perimeter Protection

Port	Proto	State	Service
5353/UDP	UDP	Open	mDNS/Zeroconf

Security Notes

- **Firmware 1.7.4** — worth checking against the [Shelly changelog](#) to confirm this is the latest Gen 2 release. Shelly has issued security patches in recent versions.
- **Port 80 filtered** — good, the web UI isn't directly accessible from this network segment. Keep it that way.
- **mDNS (5353/udp) is open** — this is how Home Assistant discovers and communicates with it. Normal and expected, but it does advertise the firmware version to anyone on the local network.
- **No MQTT, no CoAP, no UPnP** exposed — clean footprint for an IoT device.

Perimeter Protection

Example of pentest result with AI, host not protected by firewall

Prompt: scan 192.168.13.101

Here are the results for **192.168.13.101**:

Host Info

- **Hostname:** home-assistant
- **OS:** Linux 5.x (kernel 5.0–5.14)
- **Status:** Up (2.9ms latency)

Open Ports (1–1000)

Port	Service	Details
22/tcp	SSH	OpenSSH 10.2 (protocol 2.0)
53/tcp	DNS	NLnet Labs NSD
111/tcp	RPCbind	v2–4 (RPC #100000) — also on UDP

Notes

- This is your **Home Assistant** server.

Perimeter Protection

- **Port 22 (SSH)** — OpenSSH 10.2 is quite recent, good. Make sure password auth is disabled and only key-based login is allowed.
- **Port 111 (RPCbind)** — This is worth noting. RPCbind can expose information about running RPC services and is generally not something you'd want publicly accessible. On a local/VLAN-segmented network it's lower risk, but consider firewalling it if it's not needed.
- **Port 53 (DNS)** — Same NLnet Labs NSD as the UniFi device; likely part of your internal DNS setup.

Home Assistant's typical web UI (port **8123**) wasn't found in the 1–1000 range. Would you like me to scan a wider port range or check specifically for port 8123?

Prompt: test 8123

Port 8123 is open and confirmed as **Home Assistant**:

Home Assistant Web UI — Port 8123

Detail	Value
Port	8123/TCP — Open
Service	HTTP — aiohttp 3.13.3
Runtime	Python 3.14
Title	Home Assistant
robots.txt	Disallows /

Notes

Perimeter Protection

- Running on **aiohttp 3.13.3 / Python 3.14** — very recent versions, good sign.
- The UI is served over **plain HTTP** (not HTTPS). If this is accessible outside your local network or VLAN, that's a risk — credentials and tokens would be transmitted unencrypted. Consider putting it behind a reverse proxy with TLS (e.g., your nginx on the UniFi device, or a dedicated proxy).
- `robots.txt` blocking / is just cosmetic — it doesn't provide any security.

Would you like me to run a Nikto scan against port 8123 as well?

Prompt: yes

Nikto is unable to connect to port 8123, even though Nmap confirmed it's open. This is likely because:

- **Home Assistant actively blocks or drops connections** from unknown/automated scanners — it has built-in brute-force and abuse protection that can reject rapid or unusual HTTP requests.
- **Nikto's version is outdated** (flagged in every run), which may affect its ability to handle aiohttp/modern HTTP stacks correctly.
- There may be a **firewall rule or rate limiter** on the HA instance blocking the scanner's source IP after the Nmap probe.

What you can do instead

- **Manual review** — browse to `http://192.168.13.101:8123` and check the HA Security settings under *Settings* → *System* → *Security*
- Key things to verify in Home Assistant:
 - **HTTPS enabled** (via the `homeassistant` or a reverse proxy config)
 - **Trusted networks** properly scoped
 - **Multi-factor authentication** enabled for your user accounts
 - **Exposed to the internet?** — if so, ensure it's behind Nabu Casa or a properly configured reverse proxy with TLS

Perimeter Protection

Would you like me to try a Gobuster directory scan against it instead, or move on to scanning another host?

Prompt: yes

Gobuster is up and running. Here are the results:

 Gobuster Directory Scan — `http://192.168.13.101:8123`

Path	Status	Notes
/app	200	App section
/config	200	⚠ Configuration panel
/home	200	Dashboard
/history	200	Entity history/logs
/map	200	Device tracker map
/profile	200	User profile
/security	200	⚠ Security settings

 Notes

- All paths return **200 OK** with identical size (5948 bytes) — this is typical of Home Assistant's **single-page app (SPA)** design, where the frontend router handles all paths and the server returns the same `index.html` shell regardless of route.
- `/config` and `/security` are the most sensitive routes; make sure MFA is enforced.

The biggest remaining concern is still the **lack of HTTPS** on port 8123. Want me to scan any other hosts on your network?

Perimeter Protection

Firewall rules

✓ Classification:

- Access rules: Rules that allow, block, or reject access to ports, protocols, or addresses (*stateless*)
- Stateful filtering rules: allows or disables connection.
- NAT rules: Rules that allow you to "transform" one IP into another.
- Traffic characteristics change rules: Allow bandwidth according to the censor rules (e.g. QOS, *traffic shapping*). These rules are not "all or nothing".

Access and filtering rules

✓ *Checklist* for rule construction (SANS Institute):

- Anti-spoofing *rules*: Does not allow private networks coming from abroad
- User rules. Allow connections to services

Perimeter Protection

- Management rules. Allow connections to service protocols (SNMP, OSPF)
- Noise rules. Limit protocols such as OSPF outside the scope of desired networks.
- Alert rules: Alert the manager of a particular connection attempt.
- Log Rules: Placing the Connection Attempt in a Log
- ✓ Each zone (interface => network) has specific and/or shared rules. As a rule, each zone, when created, blocks all traffic
- ✓ NAT rules are checked before (when there is a server on the LAN) or after firewall rules (when there is a client on the LAN), that is, great care must be taken with the sequence.
- ✓ Firewall rules are sequentially scanned from top to bottom. Rules are applied in group order:
 1. System rules. For example, rules that ensure that we are not left without access to the firewall.

Perimeter Protection

2. Floating rules: Rules that are independent of the firewall zone.
 3. Interface groups. Set of interfaces that have been grouped together.
 4. Interface rules. Rules only applied to one interface
- ✓ The rules, as a rule, are *stateful*. This way, if a connection remains, the rule is not checked again.
- If the rule is *stateless*, all packets are confronted with the rules
 - If the rule is stateless, the order of the packet and the *timestamp of* the same are not checked => we may have an IP *spoofing problem* (what is this??)
 - IP *spoofing*: Changing the source IP address in *the packet* header
 - IP *spoofing* used in DDoS attacks (what is this??) where the IP of the source is that of the victim.
 - DDoS attacks aim to "bring down" the victim's network by not allowing it to operate normally. This is carried out with thousands of customers

Perimeter Protection

- ✓ The rules have actions:
 - *Pass*: Allows traffic.
 - *Block*: Doesn't allow traffic and doesn't warn the client that it has been blocked.
 - *Reject*: Does not allow traffic and warns the client. Very little used because we don't want them to know about our existence.
- ✓ IPV4 and IPV6 rules are usually different (often duplicated) but can be combined (one rule for both versions).
- ✓ Rules have a destination network/address.
 - Do not confuse WAN network with the internet, the WAN network is simply the interface with the public IP of the firewall. Allowing traffic to the WAN interface does not allow traffic to the internet.
- ✓ Rules have a source network/address. Usually the network for which we are creating the rule.

Perimeter Protection

- ✓ Rules can be inbound (IN) from the source to the firewall or outbound (OUT) from the firewall to the destination. Ex:
 - Allow access from the 10.10.10.0/24 network to the 10.10.11.0/24 network.
IN rule on network 10.1.10.0/24 bound to network 10.1.11.0/24



- ✓ Target flipping: Rules applied to all networks not listed in the target. Ex:
 - Allow Internet Access: List of all internal networks as a destination, but reversed => all other networks in the world
- ✓ Rules in Windows Defender?: Much simpler, *inbound* and *outbound* rules by network type: *Public/Domain/Private*
- ✓ Rules in UFW Linux?: Even simpler, *inbound* and *outbound rules*

Perimeter Protection

NAT Rules

- ✓ NAT allows you to transform addresses by separating the internal network (private IP's) from the external network (public IP)
- ✓ As in the firewall there is the source and the destination. However, there is also the *redirect* (where the traffic should be directed), this fact appears when the ratio is m:n if it is 1:n it does not exist.
- ✓ Port forwarding or Destination NAT (DNAT). Connection initiated on the WAN interface, allows mapping a WAN network port to an IP/port of one of the internal networks
 - Rule applied to ports on the WAN interface
 - The target network is one of the internal networks
 - The destination IP belongs to the destination network
 - Destination ports belong to the destination IP

Perimeter Protection

- ✓ *Outbound* or *source* NAT (SNAT). Inverse rule to DNAT. Connection initiated on private IP. Typically created automatically by the firewall.
- ✓ Access rules can be mixed with *port forwarding rules*.
 - Many firewalls (e.g. ISP routers) work by not completely separating NAT rules from access rules.
 - It makes life easier for ordinary users.
 - Play *League of legends* => perform *forward* from TCP ports: 2099, 5222-5223, 8088, 8393-8400 UDP: 5000-5500, 8088.
 - As the average user thinks it takes a lot of work to create rules, *Universal Plug and Play* appeared. Applications can create NAT rules of their own authorship. It should never be used, but:
 - 1st link on google: "it is more correct not to enable UPnP".
 - 2nd google link: "Whether it's in a smart home or an office, IoT devices work best when they are interconnected with other

Perimeter Protection

nearby devices. UPnP unites all these devices without any setbacks." Fabulous, right?

Traffic shapping *rules*

- ✓ Each rule is associated with a *pipe*. A *pipe* is a description of the internet connection: Bandwidth, ping delay, packet loss, etc.
- ✓ Each type of traffic is associated with a *Flow*: A flow is characterized by the port and protocol type, e.g.: HTTP connection: port 80, TCP
- ✓ Within each *pipe* we can determine the relative weight of each *flow* created in a *Queue*, ensuring priority to the "most interesting" traffic

Auxiliary rules

- ✓ *Floating rules*: Rules that are independent of the subnet. Frees the user from having to create the same rules for each zone.

Perimeter Protection

- ✓ *Aliases*: Frees the user from having to resort to mnemonics or paper because it allows you to create common names for interfaces, networks, servers, etc.

Zone rules

- ✓ Rules applied to a set of vlan's (zone) and not just to a subnet, facilitates a view of the set and creation of rules by security level.

Perimeter Protection

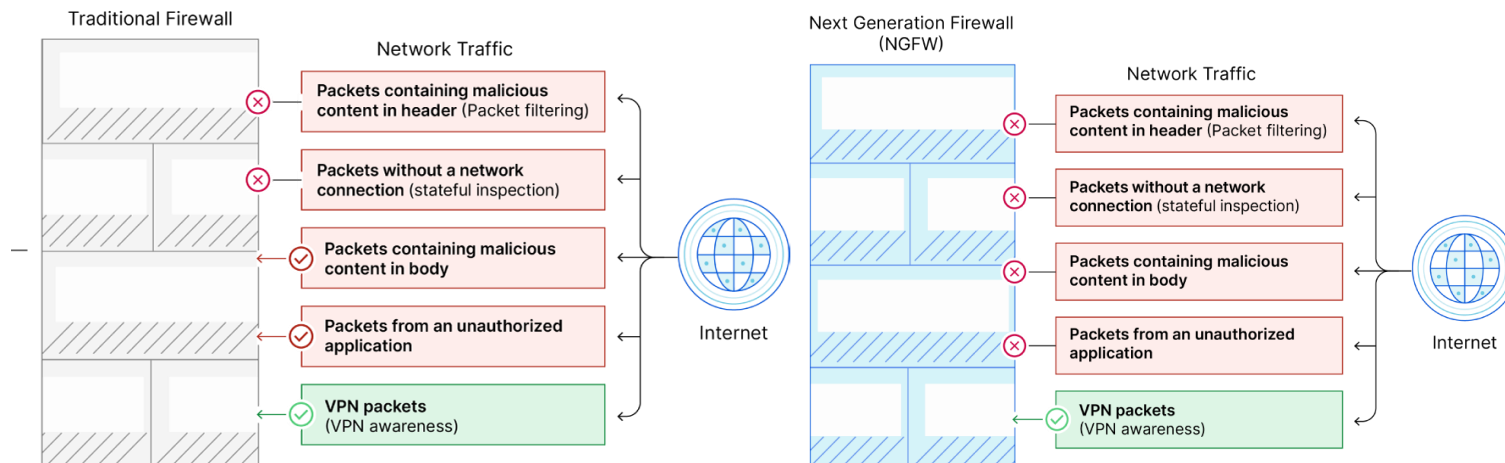
Built-in Zones		Destination Zone					
		Internal	External	Gateway	VPN	Hotspot	DMZ
Source Zone	Internal	Allow All	Policies	Allow All	Allow All	Allow All	Allow All
	External	Policies	Policies	Policies	Policies	Policies	Policies
	Gateway	Allow All	Allow All	-	Allow All	Allow All	Allow All
	VPN	Allow All	Policies	Allow All	Allow All	Allow All	Allow All
	Hotspot	Allow Return Traffic	Policies	Policies	Allow Return Traffic	Block All	Block All
	DMZ	Allow Return Traffic	Policies	Policies	Allow Return Traffic	Block All	Block All

<https://help.ui.com/hc/en-us/articles/115003173168-Zone-Based-Firewalls-in-UniFi>

Perimeter Protection

Next Generation Firewall

- ✓ There is no single definition of the capabilities of an NGFW.
 - Gartner: Adds capabilities for inspecting application behavior and intrusion prevention by often using external databases.



<https://www.cloudflare.com/learning/security/what-is-next-generation-firewall-ngfw/>

Perimeter Protection

- ✓ Not all capabilities are available on all products
- ✓ They are the largest consumer of CPU and memory in a firewall
- ✓ Many of the capabilities can be implemented separately by other products. For example: DPI with meerkat, or traffic analysis with NAGIOS.
 - All capabilities operating within a firewall may be unfeasible
 - Specialized products typically outperform.
- ✓ Capabilities are applied using policies. For example, policies for children or for adults.

Capabilities

- ✓ Application control: Verification of the type of application by inspecting the data packet or its destination, possibility to block groups of applications such as: Social networks. Possibility to block at the application level: For example Facebook

Perimeter Protection

- ✓ Cloud-resident application control: Same type of verification, but the application is resident on a server and not on the user's machine.
- ✓ Traffic analysis: Checking traffic type and quality using increasingly pattern-aware AI
- ✓ Website access control: Verification and control of access to groups of websites (with *phishing campaigns*, recent websites, social networks, etc.) or even individual websites;
- ✓ Access control to "dangerous" websites using external classifier databases (*Cloud Threat Intelligence*), for example, Bright has 4.4 billion classified URL's;
 - Access control to websites with strange behavior. The site performs unusual redirects, there is JavaScript considered malicious, etc.
 - Access control by URL
- ✓ Control of network-connected devices with group or individual policy enforcement

Perimeter Protection

- ✓ *Traffic shaping* and QOS application of differentiated bandwidth depending on the type of traffic
- ✓ Prevention of attacks with encrypted packages. For example, by prevention in the use of VPN
- ✓ *Deep packet inspection* with unencrypted or encrypted traffic (some cases as TLS)